

CS355 Lab 1 — Batch B (15:30–16:55)

Language: Scheme (DrRacket, `#lang sicp`)

Q1 (1 mark) — `median-of-three` (file name: q1.scm)

Write a procedure `median-of-three` that takes three numbers `a b c` and returns the **middle** value (i.e., neither the largest nor the smallest). You do not need recursion for this — `cond/if` and comparisons are enough.

```
(median-of-three 1 2 3)  => 2
(median-of-three 5 2 3)  => 3
(median-of-three 4 4 4)  => 4
(median-of-three -1 -2 -3) => -2
```

Q2 (1 mark) — `reverse-number` (file name: q2.scm)

Write a procedure `reverse-number` that takes a non-negative integer `n` and returns the integer formed by reversing its decimal digits. Write this as an **iterative process** — i.e., use a tail-recursive helper procedure with an accumulator. (`quotient` and `remainder` will be useful.)

```
(reverse-number 123) => 321
(reverse-number 7)  => 7
(reverse-number 120) => 21    ; the leading zero should disappear
```

Q3 (1 mark) — `repeated-apply` (file name: q3.scm)

Write a procedure `repeated-apply` that takes a procedure `f`, a non-negative integer `n`, and a starting value `x`, and returns the result of applying `f` to `x`, `n` times in a row:

```
f(f(...f(x)...))    [f applied n times]
```

This is a **higher-order function**: `f` is a procedure you're given, not one you write yourself for this part.

```
(repeated-apply (lambda (y) (+ y 1)) 5 0) => 5
(repeated-apply (lambda (y) (* y 2)) 0 7) => 7    ; applying 0 times
returns x unchanged
(repeated-apply (lambda (y) (* y 2)) 3 1) => 8
```

Take-home practice (ungraded)

people-reached-after-n-hours

A slightly-too-funny meme has just been posted in your hostel's WhatsApp group. It starts with 1 person having seen it, and every hour, everyone who has seen it forwards it onward such that the total number of people who've seen it doubles. Use your **repeated-apply** from Q3 (not a hand-written loop) to write **people-reached-after-n-hours**, which takes a non-negative integer **n** and returns how many people have seen the meme after **n** hours, starting from 1 person and doubling each hour.

```
(people-reached-after-n-hours 5) => 32  
(people-reached-after-n-hours 0) => 1
```

Once you're done, feel free to also try Batch A's Q1–Q3 for extra practice — same topics, different problems.